Figaro™: Compositional Market Design

A Blockchain Secured Market for Logistic Services

Fabrizio Romano Genovese & Alessandro Daliana

First version (Figaro: A Proof of Delivery Blockchain Solution to the Last Mile Problem, March 30,2022)

Revised version May 30, 2023

Contents

Contents **1**
Contributors 2
Acknowledgements 2
Chapter 1 ..... **3**
1.1 Introduction 3
1.2 A Last Mile Delivery Solution 3
1.2 Last Mile Problem Solutions Leverages the Community 4
1.3 A more ethical solution 5
1.4 Prerequisites 6
1.5 Synopsis 6
Chapter 2 ..... **7**
2.1 Protocol Overview 7
2.2 Impossibility of a Location-based Proof Of Delivery 8
2.3 Proof of Action 9
2.4 Protocol Features 10
Chapter 3 ..... **11**
3.1 Types of transactions 11
3.2 Service Request Transaction 12
3.3 Service Request Transaction - Cancellation 14
3.4 Service Staking 16
3.6 Dutch Auction 18
3.7 Beginning Service/Pickup Transaction 19
3.8 End of Service/Delivery Transaction 21
3.9 Slashing transactions 22
3.10 Token Reward Transactions 23
Chapter 4 ..... **24**
4.1 Geofencing 24
Chapter 5 ..... **25**
5.1 Protocol execution 25
Chapter 6 ..... **27**
6.1 Roadmap 27
Conclusion **29**
Compositional Market Design 29

Contributors

This work is mainly the result of many months of conversations between F. R. Genovese and A. Daliana. We were both animated by the desire of seeing blockchain evolve to something more than atomic transactions, and looked with interest at non-Defi applications and the Decentralized Anonymous Organizations (DAOs) that govern them. F. R. Genovese has a technical background, while A. Daliana has many years of expertise in distribution, logistics and insurance. Last mile delivery is a strictly production-focused, product-oriented business and so it seemed like a natural starting point to work on.

The idea of applying Proof of Stake was Alessandro's. Fabrizio found a technical way to reify it.

Acknowledgements

We would like to thank the crypto ecosystem as a whole, more particularly: Martin Worner from T - Grade and the Ethereum community for providing their support, amazing insights and food for thought.

We also want to give special thanks to Vlad Zamfir, without whom we would have never met.

Chapter 1

1.1 Introduction

Figaro™ is a blockchain-based solution to the last mile problem[1] in logistics.

In Proof of Work (PoW) blockchains, such as Bitcoin[2], the right to mint a block requires expending computational power to solve a difficult mathematical problem such as inverting a hash[3]. Providing such a solution testifies that a certain amount of energy has been spent in completing the task.

Figaro™ starts from a very similar conceptual framework that, in logistics, a proof that a given good has been transported from point A to some point B provides evidence of work, also an expenditure of energy, and can be used to earn the right to mint a reward. We call this idea Proof of Action (PoA).

In this work, we propose combining Proof of Stake, Proof of Action, and token minting to design a community-based, decentralized, self-regulating market as a solution to the last mile delivery problem.

The name "Figaro" pays homage to the main character of Gioachino Rossini's character in "The Barber of Seville", Figaro: the city factotum, running around making errands here and there for a myriad of clients.

1.2 A Last Mile Delivery Solution

In the field of logistics, the term "last mile" refers to the last leg of a supply chain. It involves transportation of goods from a hub to its final destination, usually the recipient's home or office.

Last mile has its own sort of peculiarities that set it apart from other legs of the logistics chain. Importantly, in moving products from transportation hub to transportation hub said products can be bundled together, resulting in a lower shipping cost per item.

Example 1.1.1 (Container ships). A container ship costs a lot of money to move, but it can transport many tons of goods. For instance, in shipping a container from one side of the Atlantic to the other, the shipping cost one would pay would be spread proportionally across all the items

[1] ——, (2022). "Last Mile," Wikipedia, the free encyclopedia, https://en.wikipedia.org/wiki/Last_mile_(transportation) (visited on 03/29/2022).
[2] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," www.bitcoin.org, pp. 1–9, 2008. https://bitcoin.org/bitcoin.pdf.
[3] A. Back, "Hashcash - a denial of service counter-measure," 2002.

in the container. Shipping many containers every month further lowers expense per item as one would benefit from many bulk shipping pricing options.

This is not necessarily true for the last mile problem, as recipient locations are scattered around and the storage capacity of the involved means of transportation – usually small trucks, cars, mopeds and even bikes – is limited.

Example 1.1.2 (Pizza delivery). If a manager of a Pizza restaurant needs to deliver pizza to their clients, they will rely on a delivery service of some sort, such as a rider driving a moped. Moreover, one can imagine that, on average, every client orders just two or three pizzas. In this case, many trips will have to be scheduled: the capacity of transporting many pizzas at once is hindered by the fact that

- Pizza will have to be delivered hot;
- Storage space is limited.

As such, the price per shipment for pizza delivery will probably be higher than the cost of shipping one kg/lb of flour from the manufacturer to the pizza restaurant, as in Example 1.1.1.

Another big issue of last-mile logistics is that recipients are often less reliable than big sorting hubs. Often, deliveries have to be rescheduled because the recipient is not available. Moreover, there is a high incidence of theft, especially when packages are left at the door[4].

The peculiarities of the last mile problem have been increasingly studied in the last years, thanks to the boom of shared mobility and business-to-consumer activities such as e-commerce. What makes it a difficult problem to solve is how it varies from area to area – compare for instance a huge north-American metropolis with regular, wide streets with a historic small European town where streets are narrow – and from product to product, pizzas as in Example 1.1.2 have to be delivered as soon as possible, while other goods such as consumer electronics can take longer.

1.2 Last Mile Problem Solutions Leverages the Community

In the last decade or so, with the advent of mobile technology, a new class of solutions to the last mile problem have come to light. The main idea is that in a sufficiently densely populated city the inhabitants may be interested in providing delivery services to round up their paycheck. If this turns out to be remunerative enough, they could consider making it a full time job. This is the core working model of companies such as Uber[5] and Lyft[6].

[4] M. Hicks, B. Stickle, and J. Harms, “Assessing the Fear of Package Theft,” American Journal of Criminal Justice, Jan. 4, 2021. http://link.springer.com/10.1007/s12103-020-09600-x (visited on 03/29/2022).
[5] ——, (2022). “Uber,” Wikipedia, the free encyclopedia, https://en.wikipedia.org/wiki/Uber (visited on 03/29/2022).
[6] ——, (2022). “Lyft,” Wikipedia, the free encyclopedia, https://en.wikipedia.org/wiki/Lyft (visited on 03/29/2022).

Indeed, almost anyone with a car can become a Uber driver and earn money by providing mobility services to passengers. This is made possible by the fact that almost everyone owns a smartphone now, which allows the company to monitor and manage very granularly the flow of operations. Similarly, one can earn money by delivering food from restaurants to consumer houses and offices. For decades restaurants have provided this service with in-house staff, but the presence of apps coordinating pickups from multiple locations frees the workers from being bound to one particular restaurant. Likewise, they free restaurants from having to manage the in-house delivery process.

Unfortunately, using these app-based solutions can result in a very exploitative business, for multiple reasons:

- First of all, these delivery systems rely on metrics like speed of delivery and customer reviews. For services such as TripAdvisor[7], the impact of negative reviews weighs far more on customer decision-making than the impact of positive ones. This results in a lot of undesirable practices such as food delivery operators speeding[8], putting theirs and everyone else's safety at risk in the hope of avoiding bad feedback.
- Secondly, apps such as Uber have huge running costs, both related to the infrastructure they have to maintain and to the need of maximizing profit to satisfy their shareholders. This results in the company managing the app retaining the majority of the shipment revenue, oftentimes commissions plus fees total over 30%[9] of the value of the meal.
- Third, restaurants using these apps are often obliged to use other services, like dedicated telephone numbers and point of sale registers, increasing the amount of value flowing from them to the company and squeezing margins[10].

1.3 A more ethical solution

All things considered, as often the case, an important problem has been partially solved at the expense of the poorest and the weakest. The underlying idea of Figaro™ is about decentralizing the whole last mile logistics process. People making deliveries coordinate through a decentralized, blockchain based protocol. Thereby reducing costs to a minimum.

---

[7] B. A. Sparks and V. Browning, "The Impact of Online Reviews on Hotel Booking Intentions and Perception of Trust," Tourism Management, vol. 32, no. 6, pp. 1310–1323, Dec. 2011. https://linkinghub.elsevier.com/retrieve/pii/S0261517711000033 (visited on 03/29/2022).

[8] H. Dong, S. Zhong, S. Xu, J. Tian, and Z. Feng, "The Relationships Between Traffic Enforcement, Personal Norms and Aggressive Driving Behaviors Among Normal e-Bike Riders and Food Delivery e-Bike Riders," Transport Policy, vol. 114, pp. 138–146, Dec. 2021.https://linkinghub.elsevier.com/retrieve/pii/S0967070X21002791 (visited on 03/29/2022).

[9] Juliusz Dzierlatka,"UberEats Commission: How Much Does Uber Eats Charge Restaurants?", Upmenu.com, January 31, 2023, https://www.upmenu.com/blog/uber-eats-commission/ (visited on 5/25/2023).

[10] Jaya Saxena, "Delivery Apps Aren't Getting Any Better", eater.com, May 29, 2019, https://www.eater.com/2019/5/29/18636255/delivery-apps-hurting-restaurants-grubhub-seamless-ubereats (visited on May 1, 2020).

In Figaro™, the drivers making deliveries are themselves miners: Proof of Work (PoW) is replaced by Proof of Stake (PoS) and Proof of Action (PoA), that is, a cryptographically secure proof that an item has been safely transported from point A to point B. As such, drivers are themselves miners, and mining profits – that is, service fees and token rewards – are almost entirely redistributed to the workers. Miners receive token rewards to subsidize the growth of the Figaro™ network. Moreover, on-chain data makes it possible to run a more objective reputation-based system and can allow workers to participate in a governance system.

Long story short, Figaro™ is a blockchain protocol with a DAO governance system.

1.4 Prerequisites

This document will be technical, but not overly mathematical. A general knowledge of how blockchain technology works is required, together with basic knowledge of cryptography, such as what is cryptographic signing or hashing. Throughout the paper, overly technical, or mathematical, remarks will be highlighted with an asterisk, as in:

> *Remark* 1.4.2 (An overly mathematical remark from Wikipedia). "To create signature keys, generate an RSA key pair containing a modulus, N, that is the product of two random secret distinct large primes, along with integers, e and d, such that ed ' 1(mod φ(N)), where φ is the Euler's totient function. The signer's public key consists of N and e, and the signer's secret key contains d.*
> *To sign a message, m, the signer computes a signature, σ, such that σ ' md(mod N). To verify, the receiver checks that σe ' m(mod N)."*

We will do our best to shape this work in such a way that remarks as Remark* 1.4.2 can be safely skipped without hindering general comprehension.

1.5 Synopsis

- In chapter 2 we give a basic overview of the protocol;
- In chapter 3 we list the different types of transactions in the Figaro™ protocol, and explain their role in detail.

Chapter 2

2.1 Protocol Overview

As we already stated in Chapter 1, we envision Figaro™ as a solution to the last-mile problem, that manages deliveries in a decentralized way.

Remark 2.1.1 (Actors involved). Here are the main actors in the protocol:

> **Shippers**: these are parties that need to send shipments. They send out delivery requests in the form of transactions, specifying how much they are willing to pay for their delivery;
>
> **Recipients**: these are the parties receiving the shipments;
>
> **Drivers**: these are the parties - people or drones - that pick-up the goods at point A to deliver them to point B, in exchange for the service fee and token rewards.

These actors can either be:

> **Honest**: meaning that they strictly abide by protocol's rules;
>
> **Rational**: meaning that they will prefer one legitimate protocol state more than another if this results in a better personal return;
>
> **Malicious**: meaning that they try to game the protocol to favor a particular outcome (such as double-spending a transaction).

Remark 2.1.2 (Working context). There are moreover other peculiarities to account for, which depend on the unique nature of logistic-related processes. Indeed, our protocol is:

> **Geofenced**: as we are dealing with last mile deliveries within a geographic market in which the drivers are authorized to operate. In most jurisdictions, drivers are required to seek and obtain prior authorization to operate. However, in some markets deliveries may be made by informal market participants, such as a neighbor or friend, in which case no prior administrative authorizations may not be required.
>
> **Non-Defi**: we are not interested in payments but in deliveries. As such, the only allowed forms of payments should be deposits, withdrawals, service fees and token rewards. This is crucial to avoid exploitation of the chain for means other than deliveries.

Price capped: even in the context of last mile logistics, deliveries are not all created equal. For instance, food delivery usually involves low value goods and low fees; on the other end of the spectrum we have security carriers, such as those delivering cash to banks. In this paper, we focus on the former, for several reasons:

- Smaller value deliveries are more abundant in number;
- They require less bureaucracy – e.g. delivering cash to banks is arguably more bureaucratic than delivering pizzas;
- They don't require workers to own or rent capital intensive and/or specialized vehicles for the job – e.g. an armored truck versus a bicycle/moped.

These characteristics provide a large, easy to recruit crowd to use the protocol.

2.2 Impossibility of a Location-based Proof Of Delivery

The most obvious approach to implement a decentralized solution for logistics would be to tie blockchain to a Proof of Delivery. In short, proving that a given package has gone from point A to point B should give the driver the right to the service fee and a token reward.

The problem with this is that drivers and recipients can collude, often in catastrophic ways. For instance, a malicious actor could impersonate both a recipient and a driver, and produce fake deliveries between them where no package exists and no delivery has really taken place. This would effectively translate in the malicious actor being able to completely manipulate the protocol.

One solution to this attack vector would be to resort to some cryptographic primitive to prove that a given actor is in some well-defined geographical location. In this way, a driver would be required to provide cryptographic proof of being first in location A and then in location B. Even if the driver and the shipper and recipient in A and B, respectively, collude, we would still be sure that the parcel moved from A to B.

Unfortunately, this is not possible. In their article "Position Based Cryptography"[11], the authors proved an impossibility theorem establishing how no such classical cryptographic protocol proving geographic location is possible without making strong assumptions on the actors

[11] N. Chandran, V. Goyal, R. Moriarty, and R. Ostrovsky, "Position Based Cryptography," in Advances in Cryptology - CRYPTO 2009, ser. Lecture Notes in Computer Science, S. Halevi, Ed., vol. 5677, Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 391–407. https://link.springer.com/chapter/10.1007/978-3-642-03356-8_23 (visited on 03/29/2022) (cit. on p. 7).

involved.[12] Moreover, said protocols require direct, speed-of-light communication between devices (usually through radio waves) and/or a trusted setup of verifiers.

On the other hand, we require our protocol to be run on a common smartphone with nothing more than an internet connection and no direct radio communication between driver devices.

2.3 Proof of Action

Given that location-based Proof of Delivery is not cryptographically viable, we resort to the next best thing, which we call Proof of Action (PoA). The underlying idea is that even if we cannot faithfully represent some action in cryptographic terms, we can assume that said action took time to be performed: time is much easier to represent and in this paper we give two ways of doing so, one more secure but more computationally expensive, and the other less secure but computationally cheaper.

Clearly, we are now stuck with the same problem we had before, that is, colluding parties can represent actions that did not really take place. Colluding parties can wait between processes to make their fake transactions which won't allow them to exploit the protocol more than these time delays would otherwise allow, but they will still have an advantage over honest drivers having to put in real work to perform the actions. Indeed, fake drivers won't sleep or have any expenses such as fuel, and may be able to produce fake transactions all day long.

This apparently useless addition can save the day if we notice something else: actions such as shipments are made in at least two steps, the pickup and the delivery. Schemes that minimize trust via interlocking guarantees: a shipper escrows the service fee, whereas the driver stakes funds that insure the value of the shipment. These funds are only unlocked only after the final delivery, free of all disputes, is confirmed. All sorts of failures translate to the guarantee being slashed or forfeited for one or both parties.

In this setup, a shipper may be trying to double-spend a transaction, but clearly it makes sense to do so only after delivery is confirmed, as a roll-back before pick-up would result in the driver not showing up at all, whereas a roll-back between pick-up and delivery would result in the escrowed sum being backrolled and the driver being effectively able to run away with the cargo.

As such, if we have a system in place to provide finality somewhere in time between shipment request and delivery, the double-spending problem would trivially vanish: as request becomes final, the most rational outcome for the shipper is to play along with the rules and go through with the delivery lifecycle.

[12] Interestingly, this result does not hold any more if we postulate that actors can share quantum resources. H. Buhrman, N. Chandran, S. Fehr, R. Gelles, V. Goyal, R. Ostrovsky, and C. Schaffner, "Position-Based Quantum Cryptography: Impossibility and Constructions," SIAM Journal on Computing, vol. 43, no. 1, pp. 150–178, Jan. 2014. http://arxiv.org/abs/1009.2490 (visited on 03/29/2022) This may become relevant in the future as quantum technology improves, but it is outside of the scope of this work for now.

2.4 Protocol Features

In light of these considerations, we conclude that the simplest and smartest way to implement PoA is to rely on some other finality provider, or to employ it in the context of a consensus protocol with finality. Whatever mechanism we chose, these are the properties we need:

> **Finality time shorter than average delivery time**: this effectively takes out an incentive to double-spend in light of the considerations made above. In chapter 3 we will cover the transaction structure that effectively implements our locked guarantee system.
>
> **Timestamping**: time is really important for us, as shipments – especially in the field of food deliveries – need to be precisely timed ('no one wants to receive a cold pizza'). Again we have many options to represent time, at least in approximate terms, in a blockchain context.
>
> **Representing stablecoins**: in contrast with other financially-focused blockchain projects, Figaro™ relies on a logic of production, where shippers use it to deliver goods and drivers use it to earn their daily living. It is apparent how Figaro™ basic currency must be as stable as possible to avoid price fluctuations in the driver's earnings. A lot of research has already been made and implemented in the field of stablecoin cryptocurrencies, with alternatives ranging from centralized solutions[13] to algorithmic ones[14], to a mix of the two[15].

Remark 2.4.1 (Figaro™ as a boost for stablecoin innovation). The geographic nature of Figaro™ may provide a boost for the development of stablecoins based on currencies different from USD. Euro and GBP based stablecoins already exist, but have not seen much adoption since the financial system is largely dollar-centric. On the contrary, it is reasonable to assume that workers working in a particular country want to be paid in that country's native currency. This will most likely include Figaro™ drivers.

Remark 2.4.2 (Figaro™ with a native token). While the benefits of using a stablecoin are clear, they are also limiting. A Figaro™ native token provides the protocol with a much greater canvas on which to design. A native token allows the protocol to support market development by subsidizing the cost of the network. Not only will the value of the token fluctuate within the confines of a specific market but, as the protocol gains adoption and new markets are created, value can flow across markets.

---

[13] Centre. (2018). "Centre Whitepaper,", https://f.hubspotusercontent30.net/hubfs/9304636/PDF/centre-whitepaper.pdf.
[14] Maker DAO. (2017). "DAI Whitepaper," https://makerdao.com/en/ whitepaper/.
[15] Frax Finance. (2022). "FRAX Whitepaper," https://docs.frax.finance/overview.

Chapter 3

3.1 Types of transactions

Whereas financial operations, such as sending money, can be considered atomic – they happen in one go – logistics is a process: a request for a service is broadcast, it is assigned to a driver, who picks up and delivers the goods, and is then paid. Furthermore, any dispute over the condition of the delivery must be resolved directly by the parties. In a decentralized context, we want to minimize trust as much as possible, and scrapping one of these steps immediately results in a bad system of incentives, such as the driver being able to steal the merchandise.

For these reasons, Figaro™ has different types of transactions that give rise to an interlocked system of guarantees. Whereas some transactions can be called ‘as-is’, other transactions need to refer to previous transactions to be considered valid. The dependence graph between different types of transactions is shown in Figure 3.1, where the arrow means ‘must refer to’.

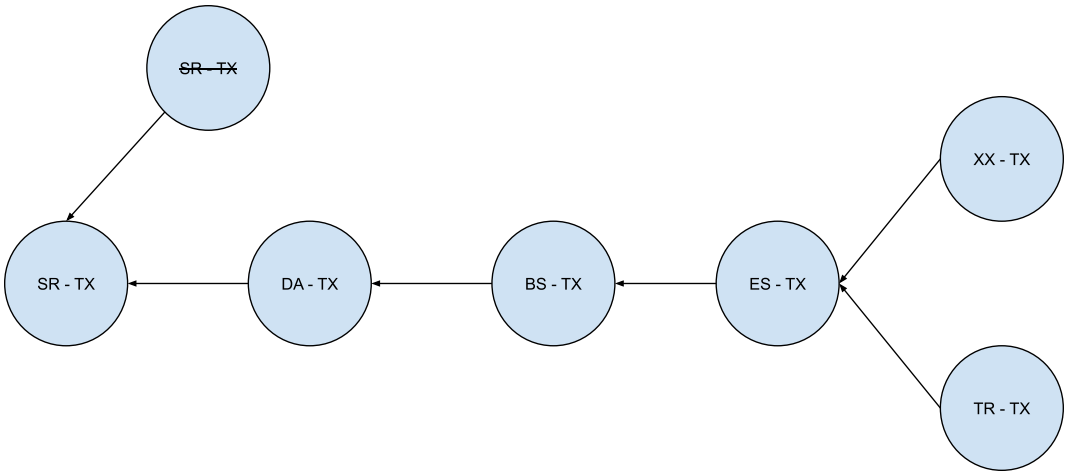


Figure 3.1: Basic Figaro™ transactions and their dependencies: SR - TX, service request transaction, section 3.1; ~~SR - TX~~, cancel service request transaction, section 3.2; DA - TX, Dutch Auction, section 3.3; BS - TX, begin service (pick-up) transaction, section 3.4; ES - TX, end service (delivery) transaction, section 3.5; XX - TX, slashing transaction, section 3.X; TR - TX, token reward (token reward minting) transaction, section 3.6

Notation 3.1.1 (Dramatis personæ). In this chapter, we will refer to the various actors involved as follows:

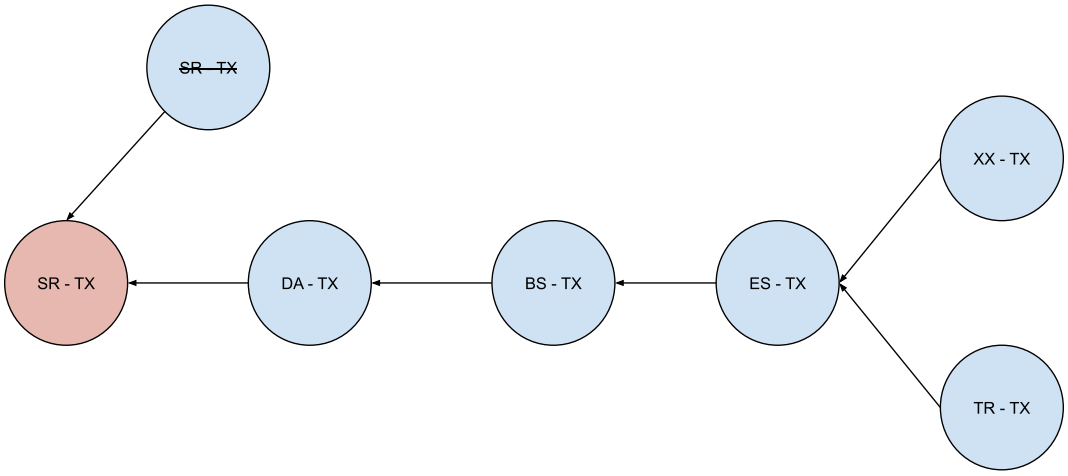
Shipper: the individual shipping some good;
Recipient: the individual receiving some good;
Driver: the individual taking charge of the delivery;

We will often refer to these actors with their proper names, as in 'driver does this, then shipper does that'.

We will moreover use object notation to specify nested properties: If A has some subproperty B, we will write A.B to denote 'property B of A'. In particular, we will write A.signee to denote the wallet that cryptographically signed A.

Remark 3.1.1 (Counting time). As we remarked in section 2.2, we need a notion of timestamping to formalize many of the concepts we need. Abstracting from the particular protocol we chose, we denote the time of any transaction in ticks. This is an integer that can represent the number of blocks after which an event occurs or a transaction is recorded, an actual timestamp or something else, depending on the mechanism chosen.

3.2 Service Request Transaction



Service request transactions represent requests for a shipment. These transactions are sent by the Shipper. They contain two monetary values - how much the Shipper is willing to pay for the service (ServiceFee) and the value of the goods (CargoInsurableValue) - plus the information the driver requires to evaluate the service: vague location data (Origin and Destination), pick-up and delivery times (RequestedPickupTime and RequestedDeliveryTime). The ServiceProvider uses this information to evaluate the offer and place their bid for the exact pick-up and delivery locations.

Service requests also include other information necessary for the Dutch auction to run: the time the auction should run (AuctionTime) and links to the exact pick-up and delivery addresses (OriginLink and DestinationLink).

Definition 3.2.1 (Request transaction). A request transaction, also denoted SR-TX, is composed of the following fields:

Shipper: the Shipper's wallet address;
Recipient: the Recipient's wallet address;
Origin: an integer, expressed in decimal grades[16]. Represents the GPS coordinates for the pick-up address;
Destination: an integer, expressed in decimal grades[3]. Represents the GPS coordinates for the delivery address;
OriginLink: a hash pointing to the complete human readable pick-up address;
DestinationLink: a hash pointing to the complete human readable delivery address;
CargoInsurableValue: an integer > 0 representing the cargo's insurable value;
ReqestedPickupTime: an integer > 0 representing the desired pick-up time (ticks);
RequestedDeliveryTime: an integer > 0 representing the desired delivery time (ticks);
ServiceFee: an integer > 0 representing the price shipper is willing to pay for service;
AuctionTime: an integer > 0 representing the shipper's auction time for locations (ticks);
Nonce: a positive integer, ensuring that the same transaction is not repeated.

To be valid, the transaction must satisfy the following conditions:

- Shipper and Recipient must be valid addresses;
- Origin and Destination are mandatory and must lie within the geofenced market;
- OriginLink and DestinationLink are mandatory and must link to the exact pick-up and delivery addresses;
- CargoInsurableValue is mandatory;
- ServiceFee is mandatory;
- RequestedPickupTime and RequestedDeliveryTime are mandatory;
- AuctionTime is mandatory;
- AuctionTime < RequestedPickupTime < RequestedDeliveryTime;
- The Nonce must be a number greater than the lastNonce, where lastNonce is the nonce of the last transaction signed by the Shipper.signee. lastNonce is considered to be 0 if no such transaction exists;
- Shipper.signee must have at least the value of the ServiceFee plus network fees in their wallet.

The transaction has the following effects:

- Locks ServiceFee in escrow;
- Locks OriginLink and DestinationLink in smart contract;
- Sets AuctionTime for Dutch Auction; and
- Provides the driver with all the necessary data to evaluate the service request and make a bid.

[16] Wikipedia Contributors. (2022). "Decimal Degrees," Wikipedia, the free encyclopedia, https://en.wikipedia.org/wiki/Decimal_degrees (visited on 03/30/2022).

Remark 3.2.1 (The role of Origin and Destination). Origin and Destination are approximate locations to protect shipper and Recipient's privacy, and need only be accurate within a radius of *n* meters.

Remark 3.2.2 (The role of OriginLink and DestinationLink). OriginLink and DestinationLink are the exact pick-up and delivery addresses. They are only disclosed to the auction winner. When the Service Request transaction is sent to the protocol these links are wrapped in a hash making them only accessible to the driver with the key. The auction winner is the only actor receiving the key. A key is generated for each link, as this information is provided at two different times: pickup address upon winning the auction and delivery address upon picking up the shipment.

Remark 3.2.3 (The role of CargoInsurableValue). CargoInsurableValue is used by the driver to determine their bid. It is also used by the protocol to manage the individual driver's load limitation: CargoInsurableValue cannot exceed the driver's stake. For management and governance purposes, the utility rate (CargoInsurableValue/Stake) can be calculated for the individual driver and the market as a whole, respectively.

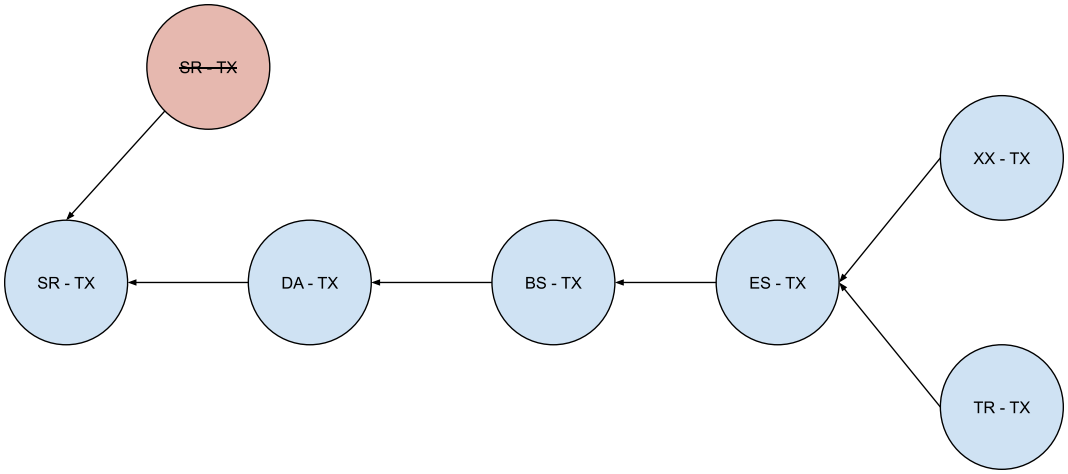
Remark 3.2.4 (The role of RequestedPickupTime and RequestedDeliveryTime). The variables RequestedPickupTime and RequestedDeliveryTime provide two deadlines for the driver. These variables are used for reputational, and quality control, purposes. Comparing requested times with actual timestamps provides an objective measure of the driver's reputation. At the market level, this data can be aggregated for community quality control and governance purposes.

Remark 3.2.5 (The role of ServiceFee). The ServiceFee is the price the shipper is willing to pay for the goods to be picked up and delivered. It is the starting price for the Dutch Auction.

Remark 3.2.6 (The role of AuctionTime). The Auctiontime sets the duration of the Dutch auction and, more importantly, signals the shipper's level of urgency. A shorter than "normal" AuctionTime will signal to drivers the shipper's urgency, while a relatively longer time will signal the opposite.

As with the ServiceFee, the history of AuctionTime over the last *m* epochs can inform the shipper's service request.

3.3 Service Request Transaction - Cancellation



A service request - cancellation transaction (~~SR - TX~~) represents an order cancellation. They can only be used before the blind Dutch Auction terminates. This signals the Shipper's change of mind.

Definition 3.3.1 (Cancel request transaction). A cancel service request transaction, also denoted ~~SR - TX~~, is composed of the following fields:

> **SR - TX**: the hash of a SR - TX;
> **Nonce**: a positive integer, ensuring that the same transaction is not repeated.

To be valid, the transaction must satisfy the following conditions:

- SR - TX must be the hash of a valid SR - TX transaction;
- No other valid transaction must point to SR - TX;
- The Nonce must be a number greater than the lastNonce, where lastNonce is the nonce of the last transaction signed by the Shipper.signee. lastNonce is considered to be 0 if no such transaction exists;
- The signee must be SR - TX.Shipper.

The transaction has the following effects:

- Returns the ServiceFee (minus gas fee) to SR - TX.Shipper;
- Cancels all auction bids;

Remark 3.3.1 (No-pointing condition). In Definition 3.3.1 we require that no other transaction be pointing to SR - TX. This is to avoid cancellation of a SR-TX that is either already canceled or already attributed to a driver.

3.4 Service Staking

Staking is one of the cornerstones of the Figaro™ Protocol. Each driver joining the market must stake an amount of value, either in stablecoin or using the Figaro™ native token. This value represents the maximum value of goods the driver can insure at any one point in time. The protocol uses this information to verify that no driver handles more merchandise than the value of their stake.

Definition 3.4.1 (Driver/Service Staking transaction). A Driver/Service Staking transaction, also denoted SS - TX, is composed of the following fields:

ServiceProvider: the driver's account;

ServiceStake: an integer > 0, this is the value of the goods the driver is able handled;
ServiceGeoFence: an integer >0 or a SBT, this is the geographic market within which the driver is authorized to operate;
Nonce: a positive integer, ensuring that the same transaction is not repeated.

To be valid, the transaction must satisfy the following conditions:

- ServiceProvider/Driver account must be valid;
- ServiceStake is mandatory and must be valid;
- ServiceGeoFence is mandatory and must be valid;
- The Nonce must be a number greater than the lastNonce, where lastNonce is the nonce of the last transaction signed by the Driver.signee. lastNonce is considered to be 0 if no such transaction exists;
- Driver.signee must have at least the value of the ServiceStake in their wallet.

The transaction has the following effects:

- Establishes the driver's stake in a specific geofenced market;

Remark 3.4.1 (The role of ServiceStake). The amount a driver stakes in the network reflects the insurable value of goods they are able to handle. The protocol can verify and control they do not surpass their stated limit before attributing a new job to a driver.

The sum of the stakes of individual drivers operating within a specific market indicates the size of the market.

For drivers using the Figaro™ native token, the utility rate of the market determines the size of the token rewards all drivers receive for successfully completing a job. The token reward for good service acts as a subsidy for the market to grow.

Remark 3.4.2 (The role of ServiceGeoFence). ServiceGeofence are variables used to define the market within which the driver is authorized to operate.

3.5 ServiceBid

Using the on-chain information provided in the service request (SR - TX) each driver evaluates the job to be done and sends in their auction bid. Bids are encrypted before the Dutch Auction so that other drivers cannot see the bid; a blind Dutch auction.

Definition 3.5.1 (ServiceBid transaction). A ServiceBid transaction, also denoted SB - TX, is composed of the following fields:

> **ServiceProvider**: the driver's account;
> **ServiceBid**: an integer < 0, this is the value of the bid the driver is making to provide the service;
> **Nonce**: a positive integer, ensuring that the same transaction is not repeated.

To be valid, the transaction must satisfy the following conditions:

- ServiceProvider account must be valid;
- ServiceBid must be valid according to the following conditions:

$$ServiceProvider.ServiceBid <= DA.SR.ServiceFee$$

$$ServiceProvider.ServiceGeoFence = DA.SR.Origin + DA.SR.Destination$$

$$ServiceProvider.ServiceValue - (\sum ServiceProvider.CargoInsurableValue\ DA - TX + \sum ServiceProvider.CargoInsurableValue\ BS - TX + \sum ServiceProvider.CargoInsurableValue\ \ ES - TX) - DA.SR.CargoInsurableValue <= 0$$

- The Nonce must be a number greater than the lastNonce, where lastNonce is the nonce of the last transaction signed by the Shipper.signee. lastNonce is considered to be 0 if no such transaction exists;
- ServiceProvider.signee must have at least the value of the ServiceBid in their wallet.

The transaction has the following effects:

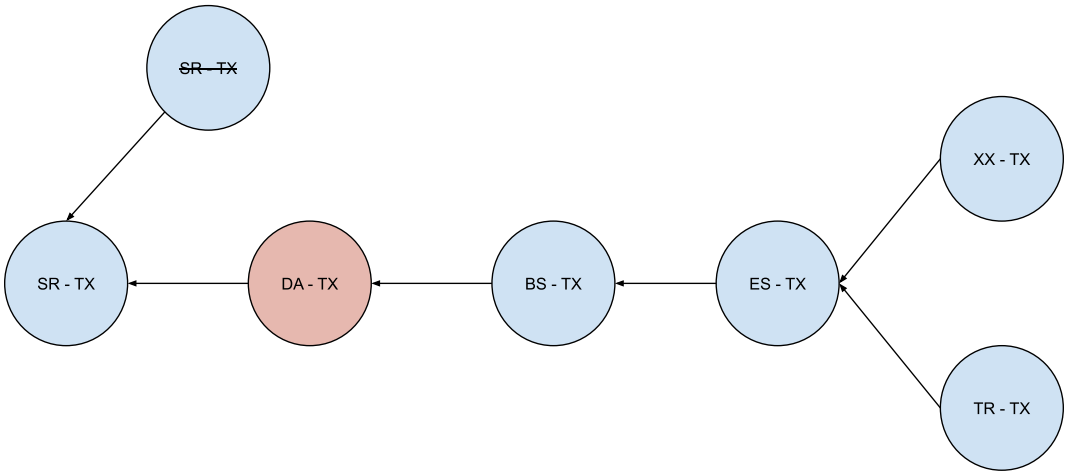
- Verifies the bid is a valid amount;
- Verifies the bid is within the driver's geofenced market;
- Verifies the additional CargoInsurableValue of this new delivery does not exceeded the driver's insurable limits;

Remark 3.5.1 (the role of variables in protocol execution) As stated above, the service request the driver is bidding on must not fall outside the driver's market, and financial. Each condition stated above should be met before the bid transaction is sent to the auction.

Remark 3.5.2 (the role of UI in bidding process) The wallet UI runs these verifications before the driver can sign, encrypt and send the bid to the auction. This apriori control should cut down on drivers spamming the auction with numerous bids.

As a fail safe, the protocol will also perform these verifications on the winning bid, eliminating those that fail in favor of other driver bids.

3.6 Dutch Auction



A dutch auction is a reverse auction, the lowest that wins. The auction process starts with the shipper's ServiceFee price. At the end of the AuctionTime, the smart contract opens the blind bids and selects the lowest bid as the winner.

A dutch auction should produce the most optimal outcome in every market situation. In non-competitive markets, drivers will execute the delivery more out of personal relations than money; bids equal to, or just below, the shipper's ServiceFee will win. In competitive markets, drivers will compete for the delivery job resulting in a price well below the shipper's ServiceFee. Ordinarily, the driver closest to the pick-up point, with enough fuel, should post the lowest bid.

Definition 3.6.1 (ServiceBid transaction). A Dutch Auction transaction, also denoted DA - TX, the conditions are:

- ServiceBid <= ServiceFee.

To be valid, the transaction must satisfy the following conditions:

- ServiceBid <= ServiceFee.

ServiceProvider.ServiceGeoFence = DA.SR.Origin + DA.SR.Destination

ServiceProvider.ServiceBid <= DA.SR.ServiceFee

ServiceProvider.ServiceValue - (∑ServiceProvider.CargoInsurableValue DA - TX +
∑ServiceProvider.CargoInsurableValue BS - TX +
∑ServiceProvider.CargoInsurableValue ES - TX) - DA.SR.CargoInsurableValue <= 0

The transaction has the following effects:

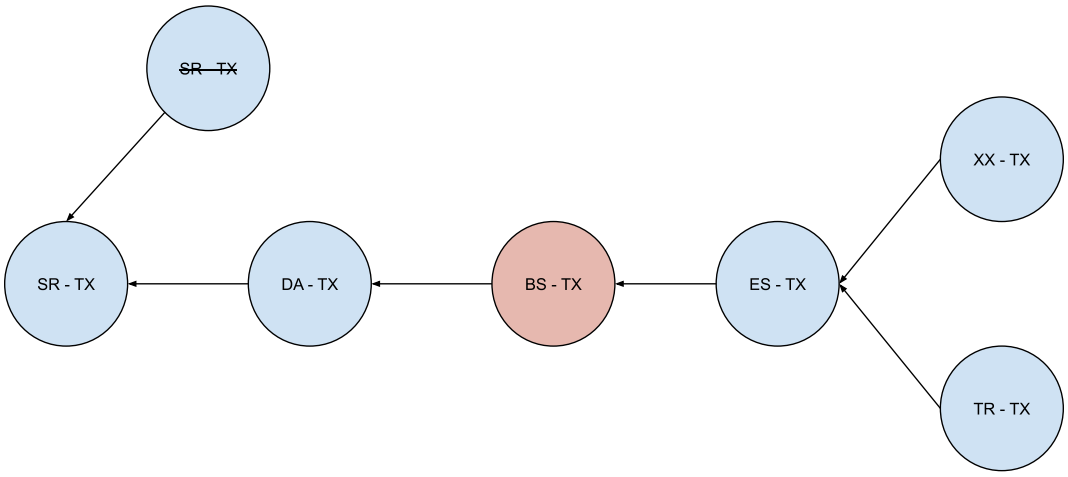
- Selects the winning bid;
- The winning bid authorizes the driver to access the OriginLink; the exact pick-up address.

Remark 3.6.1 (role of variables in protocol execution) The service request the driver is bidding on must not fall outside the driver's market, and insurance limitations.

Remark 3.6.2 (The role of the verifications). If the winning bid does not pass the protocol's verifications then the next lowest bid is evaluated, and so on until a winner is found.

Remark 3.6.3 (No Show). The auction winner is committed to providing the service. For stablecoin applications, drivers who do not show up for the pickup, will see their stake slashed by the full amount of the bid plus a penalty. The funds are transferred to the shipper as compensation and the auction starts again. For native coin applications, no penalty need be charged as the fault will be reflected in the value of the token.

3.7 Beginning Service/Pickup Transaction



The beginning service/pickup transaction registers the timestamp at which the driver effectively picks up the goods to be delivered. In doing so, the driver commits to the possibility of losing the full value of the goods being handled.

Definition 3.7.1 (Beginning Service/Pickup transaction). A beginning service/pickup transaction, also denoted BS - TX, is composed of the following fields:

- DA - TX: the hash of a DA - TX;
- nonce: a positive integer, ensuring that the same transaction is not repeated.

To be valid, the transaction must satisfy the following conditions:

- DA - TX must be the hash of a valid DA - TX transaction;
- No other transaction must point to DA - TX;
- It must be lastNonce ≤ nonce, where lastNonce is the nonce of the last transaction signed by driver.SB - TX.signee. lastNonce is considered to be 0 if no such transaction exists;
- signee must be driver.SB - TX.signee.

The transaction has the following effects:

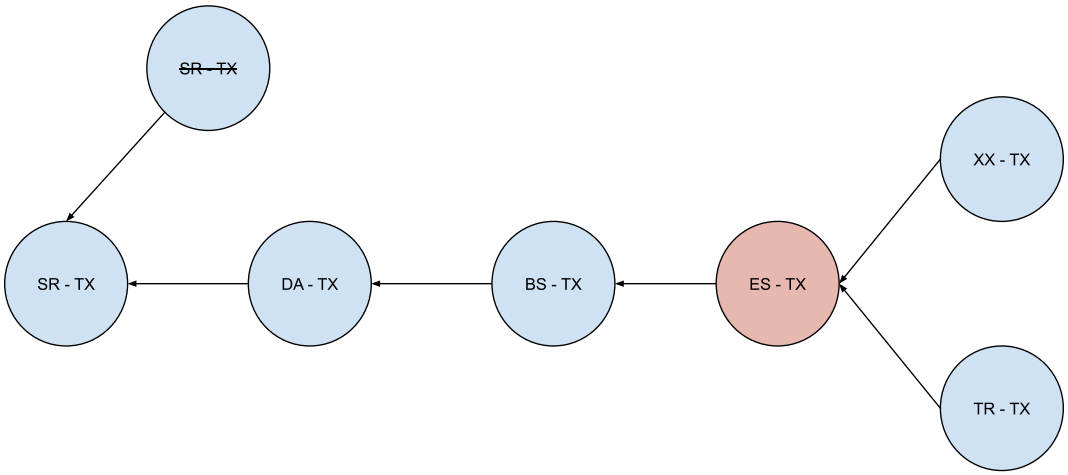
- The driver is granted access to DestinationLink.

Remark 3.7.1 (A word of caution about handing packages). As a BS - TX is signed only by the driver taking charge of the delivery, the shipper should wait for the BS - TX to be confirmed before handing the package to the driver.

Remark 3.7.2 (signee has no interest in bad behavior). In contrast to Remark 3.5.2, we notice that the signee of a SB - TX has no interest in signing a BS - TX without first having taken possession of the package, as a BS - TX will eat into the number of deliveries they can handle.

Due to the driver stake, the shipper need not worry about handing over the delivery, the value of the goods will still be insured. Conversely, the driver need not worry about the delivery's value as it is included in their count starting from DA - TX; it is already counted.

3.8 End of Service/Delivery Transaction



End of Service/delivery transactions represent the conclusion of the delivery process. It signals the fact that a given package has been handed over to its final recipient.

Definition 3.8.1 (End of Service/Delivery transaction). A end of service/delivery transaction, also denoted ES - TX, is composed of the following fields:

- BS - TX: the hash of a BS - TX;
- nonce: a positive integer, ensuring that the same transaction is not repeated.

To be valid, the transaction must satisfy the following conditions:

- BS - TX must be the hash of a valid BS - TX transaction;
- No other transaction must point to BS - TX;
- It must be lastNonce ≤ nonce, where lastNonce is the nonce of the last transaction signed by ES - TX.signee. lastNonce is considered to be 0 if no such transaction exists;
- signee must be either Shipper or Recipient;

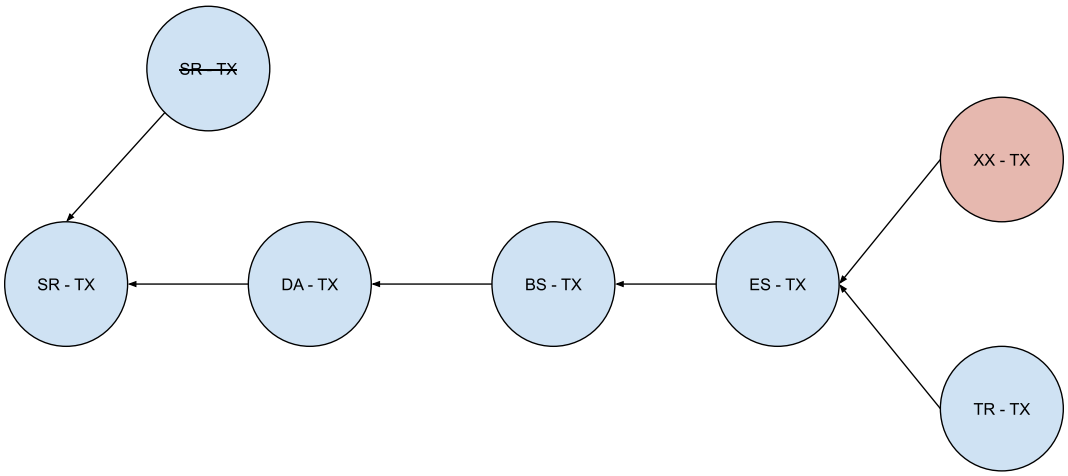
The transaction has the following effects:

- The escrowed ServiceFee is used to pay driver.Servicebid.
- The difference between shipper.ServiceFee and Driver.ServiceBid is refunded to Shipper;

Remark 3.8.1 (Delivery Disputes). It may happen that the Recipient is not satisfied with the delivery service. In this case, a dispute between driver and Recipient may arise. These two parties may come to some form of agreement where the driver partially reimburses the Recipient for the trouble caused. Any refund is organized directly between the parties outside of

the protocol. If the dispute is not resolved the driver risks forfeiting the whole amount, CargoInsurableValue.

3.9 Slashing transactions



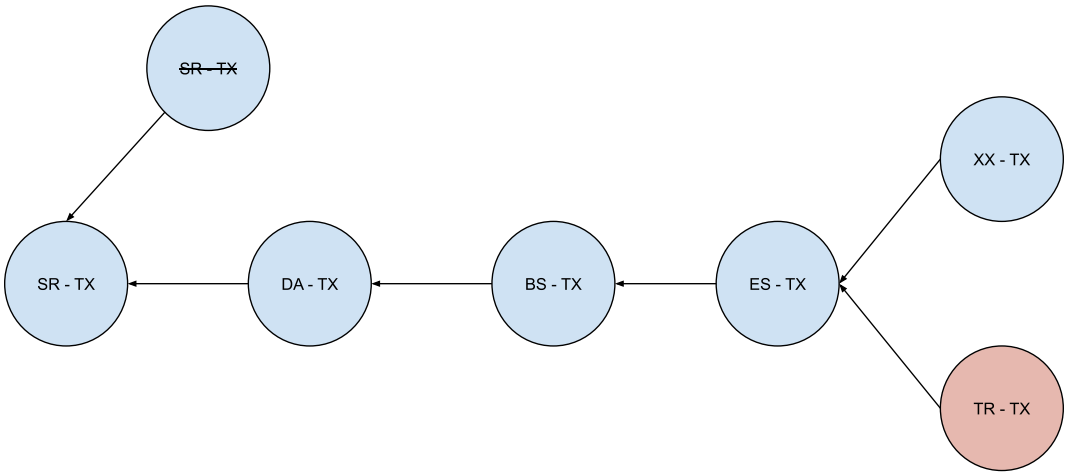
Definition 3.9.1 (Slashing transaction). A slashing transaction, also denoted XX - TX.

The attentive reader may have noted that there may be situations where things may be left "pending": For instance, a BS - TX transaction may not be followed by a ES - TX transaction. In this case, the protocol assumes the driver has made off with the goods. In the situation in which a stablecoin is used with the protocol, the protocol will then be instructed to slash the driver's stake, or ServiceStake, for the full CargoInsurableValue in their care plus a penalty. This slashed value is transferred to the shipper to compensate them for the loss. In the case of a native token, only the full CargoInsurableValue is refunded to the Shipper as the fault will be reflected in the token's value.

The slashing transaction is recorded on-chain affecting their ability to receive token rewards for *p* epochs, defined at protocol level.

The driver's loss of stake will further penalize them by reducing their ability to operate within their authorized market.

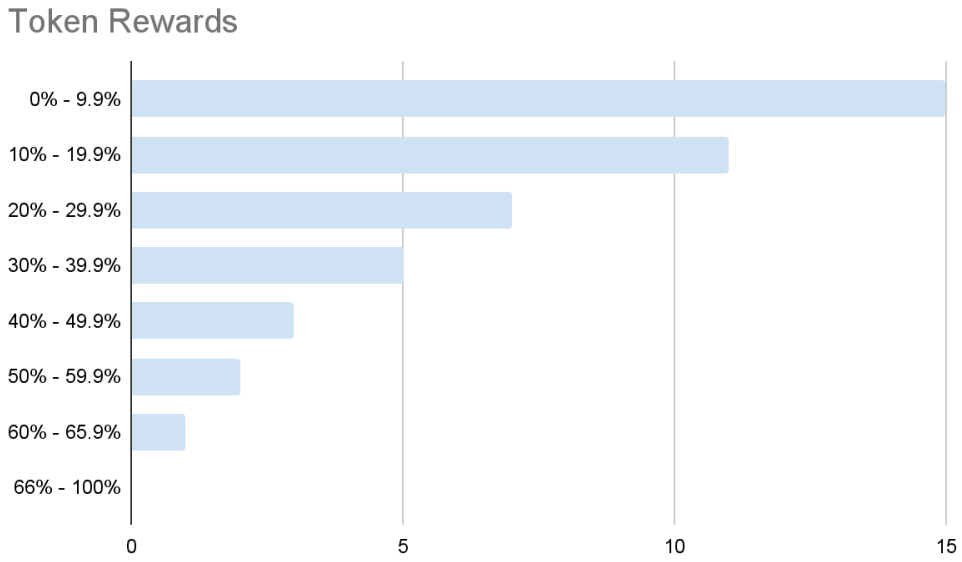
3.10 Token Reward Transactions



Establishing a network, any network, takes time and money. It is for this reason we have included in the Figaro™ Protocol design the use of a native token. A native token allows the protocol to mint a token reward which acts as a subsidy for all drivers operating in the market. This subsidy supports the market's formation over time.

Market maturity is defined in terms of its utility rate, where utility is the average value handled by the market over an epoch divided by the average value staked during that time frame, defined at the protocol level.

For example, by setting the market's utility rate at 66% the protocol will be instructed to mint rewards according to the following schedule:



Definition 3.10.1 (Token Reward transaction). A token reward transaction, also denoted TR - TX, must satisfy the following conditions:

> *For a ServiceClass in a ServiceGeoFence*
> *Calculate the average utility rate over m epochs*
> *Where utility rate = ∑ CargoInsurableValue DA - TX + ∑ CargoInsurableValue BS - TX + ∑ CargoInsurableValue ES - TX / ∑ ServiceStake <= 66%*
> *Then mint token rewards for a driver successful delivery but not those slashed*

The transaction has the following effects:

- Token rewards are minted.

Remark 3.10.1 (The role of a digressive token reward). The lower the utility rate the greater the number of reward tokens. This degressive token reward schedule should incentivize drivers, already members of a market, to persuade their colleagues, not yet members of the market, to join so they can all get more token rewards. The greater the number of drivers using the Figaro™ Protocol the faster the market will form.

On the other hand, drivers will have to be careful not to waterdown the market too much, as this increased competition will have the effect of reducing the value of their fees, and consequently the value of the tokens and the subsidy.

Chapter 4

4.1 Geofencing

Geofencing can be achieved in two ways: an Ethereum Soulbound, or non-transferrable token or Tgrade's Trusted Circles.

Remark 2.5.1 (Geofencing Mechanism). The Geofencing mechanism is an integral part of Figaro™ as it ensures the integrity of the network and links a blockchain account with an identity.

The link is important as it prevents multiple registrations from the same person[17] who may do that to flood the platform or use it to game the system to give the impression that there were many participants.

[17] Sybil Attack : https://en.wikipedia.org/wiki/Sybil_attack

There are mechanisms to support and maintain the geofencing functionality, however, there are some important decisions that need making on how the geofencing is organized. Who sets the rules? What is an acceptable proof of identity for the drivers? What do the senders and recipients need to disclose? This will vary from country to country and what identity systems are supported.

Remark 2.5.2 (Geofencing a Region). There are templates on the set-up of a Geofenced region, a model constitution covering the governance, setting who has what rights and responsibilities and on what basis.

The templates cover both a single organization and a multiple party set up wishing to create a logistics presence in a given region. This includes the governance such as the amount of escrow required for voting participants, and the quorum and thresholds for voting, required for recording the geofenced organization and participants on-chain.

To create a geofenced region, we use the Trusted Circle mechanism in Tgrade, there is a governance proposal that is submitted with the name of the group, a list of voting participants, escrow amount required for the voting participants, quorum and threshold minimums for voting.

Remark 2.5.3 (Activating Voting Participants for Geofenced Region). A voting participant is not active until they deposit the required escrow.

Remark 2.5.4 (Adding a participant). Voting participants submit a proposal to add a participant, select whether they are voting or non voting and whether the vote can pass early. The proposal is then voted on and needs to pass the quorum and threshold.

Remark 2.5.5 (Removing a Participant). A voting participant creates a proposal to remove a participant, if the participant to be removed is a voting participant the proposer has an option to return the escrow (good leaver) or slash the escrow (bad leaver) giving the proposer the option to burn the tokens or transfer them to another address. Once the proposal is submitted, the voting participants vote and it needs to pass the quorum and threshold.

Chapter 5

5.1 Protocol execution

In this chapter, we outline how transactions are executed. The attentive reader will probably have grasped already how the protocol works by just reading the transaction definitions. In any case, the life-cycle of a delivery can be seen as a ‘dependent finite state machine’ as depicted in Figure 4.1, where inputs are specified in terms of valid transactions. Here, we are using the notation input effects: the left hand side denotes the kind of transaction needed to make the

finite state machine change its state, whereas the right hand side denotes the effects that this change of state produces. In our case, the effects have been summarized for space reasons, and their full description can be found in chapter 3. The states of the finite state machine are SR (service request), DA (Dutch Auction), BS (beginning service/pick-up), ES (sending service/delivery), XX (Slashing), TR (Token reward). The special states 0 and 1 denote the initial and final states, where the delivery life cycle begins and ends, respectively.

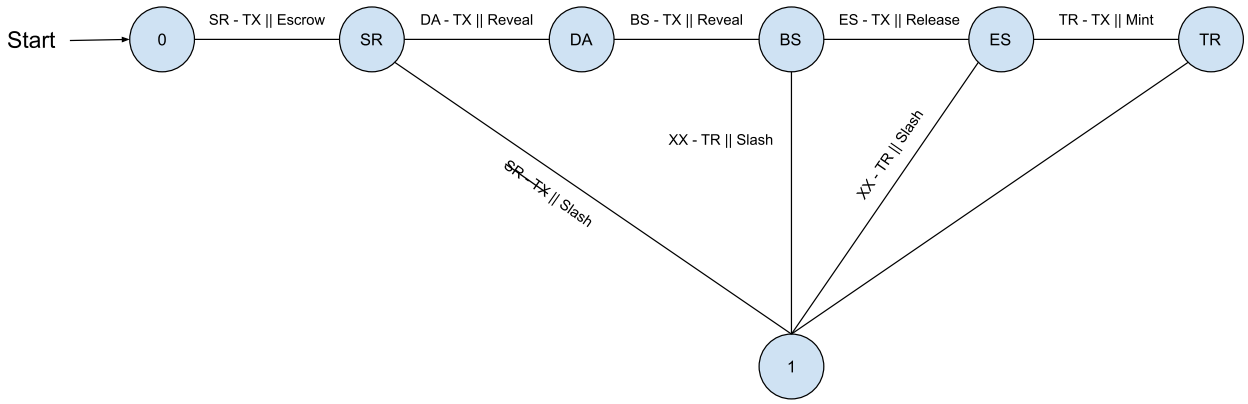


Figure 4.1: Protocol execution as a finite state machine.

Remark* 4.1.1 (Formalizing dependent finite state machines). Formally, our finite state machine is dependent, in that 'the state has state' – for instance in R we have to keep track of various parameters such as insurance and fee.

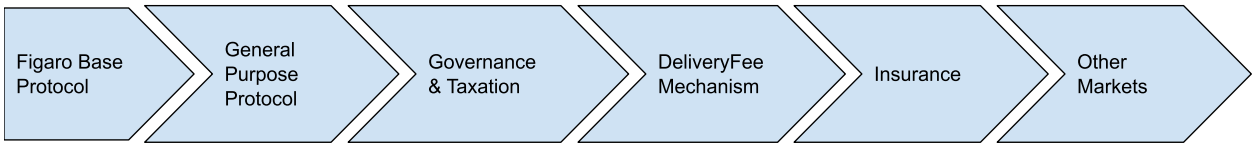
Formally, a dependent finite state machine can be realized by the following: Remembering that the execution space of a state machine is nothing more than the free category generated by its graph, dependency can be realized by considering functors from this category to the category Span of sets and spans between them. In "A Categorical Semantics for Guarded Petri Nets,"[18] we explain this process and call it 'guarded Petri nets with side-effects', but it can be readily adapted to the case at hand by relaxing the monoidality requirements.

Performing the Grothendieck construction on this functor we obtain again a free category. In short, our 'dependent finite state machine' can be compiled back to a 'state machine': the only thing we lose is the finiteness of the state space.

[18] F. Genovese and D. I. Spivak, "A Categorical Semantics for Guarded Petri Nets," in Graph Transformation, ser. Lecture Notes in Computer Science, F. Gadducci and T. Kehrer, Eds., vol. 12150, Cham: Springer International Publishing, 2020, pp. 57–74. http://arxiv.org/abs/2002.02762 (visited on 12/15/2020).

Chapter 6

6.1 Roadmap



Remark 5.1 (Transforming Figaro™ into a General Purpose Protocol). As described herein, the Figaro™ Protocol is developed for last mile deliveries. However, the concept of insurable value can be extended to almost all logistics services through the addition of a few more variables. By adding variables that are specific to the logistics industry and market segmentation Figaro™ can handle many forms of transportation and warehousing.

The following variables can be added:

- Weight and Volume
- Class of Service

Weight and volume are long standing logistical parameters which limit access to services. A vehicle can only accommodate a set volume and weight of goods before it either overflows or collapses under the weight. Warehouses have a limited surface area within which to receive, stock and ship goods.

Class of service refers to the type:

- local delivery;
    - bicycle, moped, electric scooter, motorcycle, car, light utility vehicle,...
- inter-state transport;
    - heavy duty truck, air, rail, maritime,...
- international and intercontinental transportation;
    - air, rail, maritime,...
- interplanetary transportation;
    - rocket,...
- Warehousing;
    - large central warehousing, small advanced warehousing for fast turning goods, bonded warehousing,...

A market segment is defined by the class of service offered within a geofenced location.

Remark 5.2 (DeliveryFee mechanism). Similar to the gas market for Ethereum, we expect a market to form around the ServiceFee. Much like with EIP-1559, the ServiceFee can be informed by the historic Dutch auction bids, proposed and actual, over the last *m* epochs.

At the end of *m* epoch, the Figaro™ Protocol calculates the average ServiceBid from all winning bids within the geofenced market. This value is provided to the shipper as an indication of the market rate for service. This value is only indicative as the market rate will fluctuate based on market conditions.

A spread between shipper ServiceFees and winning Driver ServiceBids can also be calculated to help shippers better price their service requests.

Remark 5.3 (Governance). As a general purpose logistics protocol, Figaro™ segments the logistics market into its many parts. Each market can have its own parameters and quality control. Using a native token, each market actor has the ability to participate in the protocol's governance. They can fork the code, add new PoA processes in between the basic operations in the base protocol; vote members in and out,...

Remark 5.4 (Taxation). One of the most burdensome administrative tasks for any business, especially for small businesses, is taxation. The Figaro™ Protocol can evolve further to automatically calculate any sales/consumption tax a driver may be obliged to collect on behalf of the state. By including a tax variable for a specific class of service conducted within a particular market the amount of tax collected can be readily calculated and sent to a separate account.

A similar treatment can be used for other taxes the driver may pay, for example: income tax, payroll taxes, service fees, apprenticeship taxes, and so on.

Remark 5.5 (Insurance). The Figaro™ protocol is designed to leverage the power of staking, which effectively acts as a form of insurance for the services rendered and to manage the driver's rate of production as well as that of the market as a whole. However, using the on-chain artifacts produced by the delivery activity there is a plethora of data that can be used to programmatically calculate the premium due to obtain the maximum insurance coverage. The fee mechanism described above can be modified to automatically pay the premium to the insurance liquidity providers, thus compensating them for the risk they are taking.

Remark 5.6 (Markets for Digital Artifacts) Like the digital artifacts for insurance, there are plenty of others which result from executing an order. A driver provides location-based data artifacts every time they drive their vehicle. These artifacts can be held in decentralized storage and monetized for the driver's benefits. The food order line items used by the restaurant to prepare the meal are also artifacts. They can be used to generate purchases from the local food purveyor, who can also use Proof of Action to integrate their processes into those of their client restaurants. The items ordered can also be the source of market information, for competitors entering the market and/or restaurants expanding into new markets. Here again, storing this

information in a decentralized database will allow the restaurant to monetize this information themselves.

These are just a few of data artifacts that can be stored and monetized in ways that benefit the data owner instead of the corporations who own the application. These should be studied and new protocols developed to manage markets for this information as well.

Conclusion

Compositional Market Design

Using the properties of the underlying blockchain network to secure the market, the Figaro™ Protocol starts with a self-regulated last mile delivery market. This market then lead to the development of other markets for other data artifacts:

- Downstream
    - Insurance;
    - Taxation;
    - GPS data; and
- Upstream
    - Types of cuisine;
    - Menu items; and
    - Food products.

Likewise, the robust nature of the protocol allows it to be extended into other logistics markets, there too with upstream and downstream consequences.

As we have shown, combining different cryptographic and blockchain primitives (Proof of Stake, Proof of Action and Public Key Infrastructure, native tokens) it is possible to compose markets that are more complex than today's atomic exchanges. And, separate markets can be combined to create entirely self-regulating economic systems.

To be more precise, using compositional market design data producers can maintain the ownership and control of their data artifacts thus allowing them to be monetized, or not, for their own benefit. This shifts value and power away from today's Web 2.0 behemoths to the individual user for a more just and ethical economic use of the Internet.